

Bima Kwik Database Design Document

This document outlines the database design for the Bima Kwik digital insurance platform. It describes the entities, attributes, and relationships within the database.

1. Introduction

The database is a crucial component of the Bima Kwik system, responsible for storing and managing all persistent data. This document details the logical database design, including entity-relationship diagrams (ERDs) and schema definitions.

2. Design Goals

- **Data Integrity:** Ensure accuracy and consistency of data through normalization and constraints.
- **Scalability:** Design the database to handle increasing data volumes and user traffic.
- **Performance:** Optimize the database for efficient data retrieval and manipulation.
- **Security:** Protect sensitive data through appropriate security measures.
- **Maintainability:** Create a clear and well-structured schema for easy maintenance and updates.

3. Technology

- **Database Management System (DBMS):** PostgreSQL 14 or later

4. Logical Design

4.1 Entity-Relationship Diagram (ERD)

[An Entity-Relationship Diagram (ERD) visually representing the entities and their relationships. Due to the limitations of text-based representation, I'll provide a textual description of the ERD. A graphical tool is highly recommended for creating and visualizing the ERD.]

Entities and Relationships (Textual Representation):

- **Users**
 - Attributes: user_id (PK), username, email, password, role_id, profile_id, created_at, updated_at
 - Relationships:
 - One-to-one with Profiles
 - Many-to-one with Roles
 - One-to-many with Policies
 - One-to-many with Transactions
 - One-to-many with Claims
 - Description: Stores user account information.
- **Roles**
 - Attributes: role_id (PK), name, description
 - Relationships:
 - One-to-many with Users
 - Description: Defines user roles (e.g., customer, agent, broker, admin).
- **Profiles**
 - Attributes: profile_id (PK), full_name, gender, dob, phone, id_type, id_number, address, region_id, district_id, created_at, updated_at

- Relationships:
 - One-to-one with Users
 - Description: Stores detailed user profile information.
- **Products**
 - Attributes: product_id (PK), name, description, type_id, premium, benefits, exclusions, coverage_area, age_range, details, created_at, updated_at
 - Relationships:
 - Many-to-one with ProductTypes
 - One-to-many with Policies
 - Description: Defines insurance products.
- **ProductTypes**
 - Attributes: type_id (PK), name, description
 - Relationships:
 - One-to-many with Products
 - Description: Defines types of insurance products (e.g., motor, health, life).
- **Policies**
 - Attributes: policy_id (PK), user_id, product_id, start_date, end_date, premium, status, details, created_at, updated_at
 - Relationships:
 - Many-to-one with Users
 - Many-to-one with Products
 - One-to-many with Claims
 - Description: Stores insurance policy information.
- **Transactions**
 - Attributes: transaction_id (PK), user_id, type, amount, status, details, created_at
 - Relationships:
 - Many-to-one with Users
 - Description: Stores financial transaction records.
- **Claims**
 - Attributes: claim_id (PK), policy_id, claimant_id, date, status, amount, details, created_at, updated_at
 - Relationships:
 - Many-to-one with Policies
 - Many-to-one with Users (claimant)
 - Description: Stores insurance claim information.
- **Reports**
 - Attributes: report_id (PK), type, date, description, data, created_at
 - Description: Stores generated reports. 'Data' field can be JSON for flexibility.
- **Documents**
 - Attributes: document_id (PK), user_id, type, path, size, name, created_at
 - Relationships:
 - Many-to-one with Users
 - Description: Stores uploaded documents.
- **ServiceProviders**
 - Attributes: provider_id (PK), name, type, location, contact_person, phone, email, status, created_at, updated_at
 - Description: Stores information about service providers.
- **Regions**
 - Attributes: region_id (PK), name, code
 - Description: Stores region information
- **Districts**
 - Attributes: district_id (PK), region_id, name, code
 - Relationships:
 - Many-to-one with Regions
 - Description: Stores district information

]

4.2 Schema Details

This section provides a more detailed description of each table in the database.

- **Users Table**

- Description: Stores user account information.
- Columns:
 - user_id (SERIAL, PRIMARY KEY): Unique identifier for the user.
 - username (VARCHAR(255), UNIQUE, NOT NULL): User's login username.
 - email (VARCHAR(255), UNIQUE, NOT NULL): User's email address.
 - password (VARCHAR(255), NOT NULL): Hashed password.
 - role_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES Roles(role_id)): Foreign key referencing the user's role.
 - profile_id (INTEGER, UNIQUE, FOREIGN KEY REFERENCES Profiles(profile_id)): Foreign key to user profile
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the user was created.
 - updated_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the user was last updated.

- **Roles Table**

- Description: Stores user roles.
- Columns:
 - role_id (SERIAL, PRIMARY KEY): Unique identifier for the role.
 - name (VARCHAR(255), UNIQUE, NOT NULL): Name of the role (e.g., 'customer', 'agent', 'admin').
 - description (TEXT): Description of the role.

- **Profiles Table**

- Description: Stores user profile information
- Columns:
 - profile_id (SERIAL, PRIMARY KEY): Unique identifier for the profile.
 - full_name (VARCHAR(255), NOT NULL): User's full name.
 - gender (INTEGER): User's gender.
 - dob (DATE): User's date of birth.
 - phone (VARCHAR(20), NOT NULL): User's phone number.
 - email (VARCHAR(255)): User's email address.
 - id_type (INTEGER): Type of identification
 - id_number (VARCHAR(255), NOT NULL): User's identification number.
 - street (VARCHAR(255)): User's street address
 - country_code (VARCHAR): User's country code
 - fax (VARCHAR)
 - region_id (INTEGER, FOREIGN KEY REFERENCES Regions(region_id)): User's region
 - district_id (INTEGER, FOREIGN KEY REFERENCES Districts(district_id)): User's district
 - postaladdress (VARCHAR)
 - losspayee (VARCHAR)
 - vrn (VARCHAR)
 - nida (VARCHAR)
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the profile was created.
 - updated_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the profile was last updated.

- **Products Table**

- Description: Stores insurance product information.
- Columns:
 - product_id (SERIAL, PRIMARY KEY): Unique identifier for the product.
 - name (VARCHAR(255), NOT NULL): Name of the insurance product.
 - description (TEXT): Description of the product.
 - type_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES ProductTypes(type_id)): Foreign key referencing the product type.
 - premium (DECIMAL, NOT NULL): Base premium for the product.
 - benefits (TEXT): Description of the product benefits.
 - exclusions (TEXT): Description of the product exclusions.
 - coverage_area (VARCHAR(255)): Geographical coverage area.
 - age_range (VARCHAR(255)): Age range for the product.
 - details (JSONB): A flexible JSONB column to store product-specific details.
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the product was created.
 - updated_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the product was last updated.
- **ProductTypes Table**
 - Description: Stores insurance product types.
 - Columns:
 - type_id (SERIAL, PRIMARY KEY): Unique identifier for the product type.
 - name (VARCHAR(255), UNIQUE, NOT NULL): Name of the product type (e.g., 'Motor', 'Health', 'Life').
 - description (TEXT): Description of the product type.
- **Policies Table**
 - Description: Stores insurance policy information.
 - Columns:
 - policy_id (SERIAL, PRIMARY KEY): Unique identifier for the policy.
 - user_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES Users(user_id)): Foreign key referencing the user who owns the policy.
 - product_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES Products(product_id)): Foreign key referencing the insurance product.
 - start_date (DATE, NOT NULL): Policy start date.
 - end_date (DATE, NOT NULL): Policy end date.
 - premium (DECIMAL, NOT NULL): Premium amount for the policy.
 - status (VARCHAR(255), NOT NULL): Policy status (e.g., 'Active', 'Expired', 'Cancelled').
 - details (JSONB): A flexible JSONB column to store policy-specific details.
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the policy was created.
 - updated_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the policy was last updated.
- **Transactions Table**
 - Description: Stores financial transaction information.
 - Columns:
 - transaction_id (SERIAL, PRIMARY KEY): Unique identifier for the transaction.
 - user_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES Users(user_id)): Foreign key referencing the user who initiated the transaction.
 - type (VARCHAR(255), NOT NULL): Type of transaction (e.g., 'Payment', 'Commission').
 - amount (DECIMAL, NOT NULL): Transaction amount.
 - status (VARCHAR(255), NOT NULL): Transaction status (e.g., 'Success', 'Pending', 'Failed').
 - details (TEXT): Details.
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the transaction occurred.

- **Claims Table**

- Description: Stores insurance claim information.
- Columns:
 - claim_id (SERIAL, PRIMARY KEY): Unique identifier for the claim.
 - policy_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES Policies(policy_id)): Foreign key referencing the associated policy.
 - claimant_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES Users(user_id)): Foreign key referencing the user who filed the claim.
 - date (DATE, NOT NULL): Date the claim was filed.
 - status (VARCHAR(255), NOT NULL): Claim status (e.g., 'Submitted', 'Processing', 'Approved', 'Rejected').
 - amount (DECIMAL): Claim amount.
 - details (TEXT): Claim details.
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the claim was created.
 - updated_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the claim was last updated.

- **Reports Table**

- Description: Stores generated reports.
- Columns:
 - report_id (SERIAL, PRIMARY KEY): Unique identifier for the report.
 - type (VARCHAR(255), NOT NULL): Type of report (e.g., 'Transaction', 'Claims').
 - date (DATE, NOT NULL): Date the report was generated.
 - description (TEXT): Description of the report.
 - data (JSONB): The report data, stored as a JSONB object for flexibility.
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the report was created.

- **Documents Table**

- Description: Stores uploaded documents
- Columns:
 - document_id (SERIAL, PRIMARY KEY): Unique identifier
 - user_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES Users(user_id)): Foreign key referencing the user who uploaded the document
 - type (VARCHAR): Type of document
 - path (VARCHAR): Path to the document
 - size (INTEGER)
 - name (VARCHAR)
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the document was created.

- **ServiceProviders Table**

- Description: Stores information about service providers.
- Columns:
 - provider_id (SERIAL, PRIMARY KEY): Unique identifier
 - name (VARCHAR): Name of the service provider
 - type (VARCHAR): Type of service provider
 - location (VARCHAR)
 - contact_person (VARCHAR)
 - phone (VARCHAR)
 - email (VARCHAR)
 - status (VARCHAR)
 - created_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the service provider was created.
 - updated_at (TIMESTAMP WITH TIME ZONE, DEFAULT now()): Timestamp when the service provider was last updated.

- **Regions Table**

- Description: Stores regions

- Columns:
 - region_id (SERIAL, PRIMARY KEY)
 - name (VARCHAR)
 - code (VARCHAR)
- **Districts Table**
 - Description: Stores districts
 - Columns:
 - district_id (SERIAL, PRIMARY KEY)
 - region_id (INTEGER, NOT NULL, FOREIGN KEY REFERENCES Regions(region_id)): Foreign key to Regions
 - name (VARCHAR)
 - code (VARCHAR)

5. Indexing Strategy

- Primary keys are automatically indexed.
- Foreign keys should be indexed to improve join performance.
- Consider indexing frequently queried columns (e.g., email in Users, status in Policies, date in Claims).
- Use composite indexes for queries that involve multiple columns.
- Regularly analyze query performance and adjust indexes as needed.

6. Security Considerations

- **Password Storage:** Store hashed passwords using a strong hashing algorithm (e.g., bcrypt).
- **Data Encryption:** Implement encryption at rest for sensitive data (e.g., PII).
- **Access Control:** Use database-level access control to restrict access to tables and views based on user roles.
- **Principle of Least Privilege:** Grant only the necessary privileges to database users and applications.
- **Regular Security Audits:** Conduct regular security audits to identify and address potential vulnerabilities.

7. Scalability and Performance Considerations

- **Connection Pooling:** Implement connection pooling at the application level to efficiently manage database connections and reduce overhead.
- **Prepared Statements:** Use prepared statements or parameterized queries to prevent SQL injection and improve query performance by reusing execution plans.
- **Efficient Queries:** Write optimized SQL queries, avoiding complex joins and unnecessary data retrieval. Use EXPLAIN to analyze query performance.
- **Appropriate Data Types:** Use the most appropriate data types for each column to minimize storage space and improve performance. For example, use SMALLINT instead of INTEGER if the range of values is small.
- **Data Partitioning:** For very large tables (e.g., Transactions, Claims), consider partitioning the data based on a relevant criterion (e.g., date range) to improve query performance.
- **Read Replicas:** Set up read replicas to distribute read traffic and improve query performance, especially for frequently read data.
- **Caching:** Implement caching mechanisms (e.g., using Redis) to store frequently accessed data in memory and reduce database load.
- **Asynchronous Operations:** Offload long-running or non-critical database operations (e.g., report generation) to a background queue using message queues (e.g., Kafka) to maintain responsiveness.
- **Monitoring:** Implement robust database monitoring to track key performance metrics (e.g., query response times, CPU usage, memory usage, disk I/O) and identify potential bottlenecks.

- **Database Sharding:** Implement database sharding if the data volume becomes too large for a single database server.
- **Data Warehousing:** Consider a separate data warehouse for reporting and analytics.
- **NoSQL Database:** Evaluate the use of a NoSQL database (e.g., MongoDB) for specific data that doesn't fit well into the relational model (e.g., session data, log data).